Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 4

Implementation & Evaluation of Support Vector Machine (SVM)

Course Code: ECE 4246

Course Title: Machine Learning Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 13, 2026

Contents

1 Theory and Introduction **1**
1.1 Binary SVM and Hinge Loss 1
1.2 Optimization via Stochastic Gradient Descent 2
1.3 One-vs-Rest (OvR) Strategy 2

2 Methodology **2**
2.1 Dataset 2
2.2 Algorithms 2
2.3 Code 2

3 Results and Discussion **5**
3.1 Quantitative Results 5
3.2 Visual Analysis 6

4 Conclusion **7**

References **7**

Implementation & Evaluation of Multi-Class Support Vector Machine (SVM)

1 Theory and Introduction

The Support Vector Machine (SVM) is a powerful supervised machine learning algorithm primarily used for classification tasks. Unlike standard classifiers that output probabilities, SVM acts as a maximum margin classifier. It maps training examples to points in space so as to maximize the width of the gap between different categories [1]. Figure 1 illustrates the core components of this architecture, highlighting the optimal decision boundary, the maximized margin, and the critical support vectors that anchor the boundary.

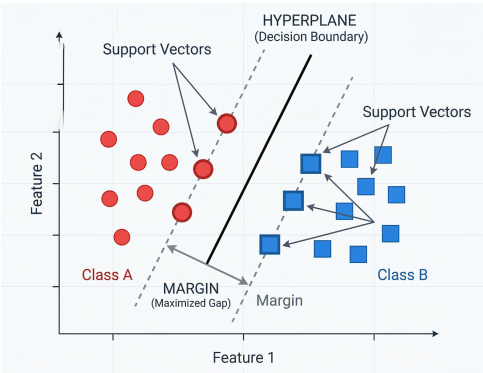


Figure 1: Conceptual illustration of a binary Support Vector Machine. The algorithm identifies the optimal hyperplane that maximizes the margin gap between Class A and Class B. The data points lying directly on the margin boundaries are the support vectors.

Standard SVMs are strictly binary classifiers. To handle multi-class datasets, meta-strategies such as the One-vs-Rest (OvR) approach must be implemented.

1.1 Binary SVM and Hinge Loss

For a binary classification task where labels $y \in \{-1, 1\}$, a linear SVM attempts to find a hyperplane $w^T x + b = 0$ that separates the classes. The objective is to minimize the Hinge Loss with L_2 regularization:

$$J(w, b) = \lambda ||w||^2 + \frac{1}{n} \sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b)) \tag{1}$$

Where λ acts as the regularization parameter controlling the margin width. A larger λ creates a wider (soft) margin, while a smaller λ enforces a strict (hard) margin.

1.2 Optimization via Stochastic Gradient Descent

The parameters are optimized using Stochastic Gradient Descent (SGD). If a sample is correctly classified and lies strictly outside the margin ($y_i(w^T x_i + b) \geq 1$), the gradient only depends on the regularization term:

$$w \leftarrow w - \alpha(2\lambda w) \tag{2}$$

If the sample is misclassified or lies within the margin, the gradient incorporates the sample's features:

$$w \leftarrow w - \alpha(2\lambda w - y_i x_i) \tag{3}$$

$$b \leftarrow b - \alpha(-y_i) \tag{4}$$

1.3 One-vs-Rest (OvR) Strategy

To classify K classes, K separate binary SVM models are trained. The k-th classifier is trained to separate the data belonging to class k (labeled as $+1$) from all other classes (labeled as -1). For a new input x, the class is assigned to the classifier that yields the highest confidence score $w^T x + b$:

$$\hat{y} = \arg \max_{k \in \{1,\ldots,K\}} (w_k^T x + b_k) \tag{5}$$

2 Methodology

2.1 Dataset

The *Iris* dataset was utilized, containing three distinct classes (*Setosa*, *Versicolor*, and *Virginica*). The dataset was standardized, and the labels were mapped appropriately for the OvR strategy. Two features (Petal Length and Petal Width) were used to allow for 2D visualization of the decision boundaries [2].

2.2 Algorithms

Algorithm 1 Multi-Class SVM (One-vs-Rest Training)

Input: Training Data (X, Y) with K classes, λ, α, *epochs*
Initialize: K instances of Binary SVMs
for $k = 1$ to K **do**
    $Y_{binary} \leftarrow \{1 \text{ if } y_i == k \text{ else } -1 \text{ for } y_i \in Y\}$
    Train Model$_k$ on (X, Y_{binary}) using Hinge Loss and SGD
end for
return array of trained models

2.3 Code

The implementation was done in Python using NumPy for matrix operations. The training loop iteratively updates the weights and bias for each binary classifier based on the computed gradients from the Hinge Loss function. The decision boundaries were visualized using Matplotlib, and the evaluation metrics were calculated using Scikit-learn's classification report and confusion matrix utilities.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# 1. EVALUATION & HELPER FUNCTIONS
```

```python
def train_test_split_custom(X, y, test_size=0.2, seed=42):
    np.random.seed(seed)
    indices = np.random.permutation(len(X))
    split = int(len(X) * (1 - test_size))
    return (
        X[indices[:split]],
        X[indices[split:]],
        y[indices[:split]],
        y[indices[split:]],
    )


def standardize(X, mean_val=None, std_val=None):
    if mean_val is None or std_val is None:
        mean_val = np.mean(X, axis=0)
        std_val = np.std(X, axis=0)
        std_val[std_val == 0] = 1
    return (X - mean_val) / std_val, mean_val, std_val


def get_confusion_matrix_3class(y_true, y_pred):
    cm = np.zeros((3, 3), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm


def evaluate_metrics_multiclass(y_true, y_pred):
    cm = get_confusion_matrix_3class(y_true, y_pred)
    accuracy = np.trace(cm) / np.sum(cm)

    precisions, recalls = [], []
    for i in range(3):
        tp = cm[i, i]
        fp = np.sum(cm[:, i]) - tp
        fn = np.sum(cm[i, :]) - tp

        precisions.append(tp / (tp + fp) if (tp + fp) > 0 else 0)
        recalls.append(tp / (tp + fn) if (tp + fn) > 0 else 0)

    macro_precision = np.mean(precisions)
    macro_recall = np.mean(recalls)
    macro_f1 = (
        (2 * macro_precision * macro_recall) / (macro_precision + macro_recall)
        if (macro_precision + macro_recall) > 0
        else 0
    )

    return accuracy, macro_precision, macro_recall, macro_f1, cm

```

```python

# 2. PLOTTING FUNCTIONS
def plot_multiclass_cost_history(models, class_names):
    plt.figure(figsize=(10, 5))
    colors = ["purple", "teal", "orange"]  # Distinct colors for the 3 curves

    for i, model in enumerate(models):
        plt.plot(
            model.cost_history,
            color=colors[i],
            linewidth=2,
            label=f"{class_names[i]} vs Rest",
        )

    plt.title("Multi-Class SVM Training: Hinge Loss vs. Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Cost (Hinge Loss)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()


def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    ax.figure.colorbar(im, ax=ax)

    ax.set(
        xticks=np.arange(cm.shape[1]),
        yticks=np.arange(cm.shape[0]),
        xticklabels=class_names,
        yticklabels=class_names,
        title=title,
        ylabel="Actual Label",
        xlabel="Predicted Label",
    )

    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(
                j,
                i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black",
                fontsize=14,
            )
    fig.tight_layout()
    plt.show()


```

```python
def plot_multiclass_decision_boundary(X, y, model, class_names, title):
    plt.figure(figsize=(10, 6))
    colors = ["purple", "teal", "orange"]
    markers = ["o", "s", "^"]
    # Create mesh
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))
    # Predict over mesh
    mesh_points = np.c_[xx.ravel(), yy.ravel()]
    Z = model.predict(mesh_points)
    Z = Z.reshape(xx.shape)
    # Plot filled contours for decision regions
    plt.contourf(xx, yy, Z, alpha=0.3, cmap="viridis")
    # Scatter actual points
    for i in range(3):
        plt.scatter(
            X[y == i, 0],
            X[y == i, 1],
            c=colors[i],
            marker=markers[i],
            edgecolors="k",
            s=60,
            label=class_names[i],
        )

    plt.title(title)
    plt.xlabel("Petal Length (Standardized)")
    plt.ylabel("Petal Width (Standardized)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()


# 3. SVM ALGORITHMS (Base & OvR Wrapper)
class LinearSVM:
    def __init__(self, learning_rate=0.001, lambda_param=0.01, epochs=1000):
        self.lr = learning_rate
        self.lambda_param = lambda_param
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []  # ADDED: Initialize cost history array

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0
        self.cost_history = []  # Reset on fit

        for epoch in range(self.epochs):
            # ADDED: Calculate Hinge Loss for the epoch history
            distances = 1 - y * (np.dot(X, self.w) + self.b)
            distances[distances < 0] = 0  # max(0, distance)
            hinge_loss = self.lambda_param * np.sum(self.w**2) + (
                1 / n_samples
            ) * np.sum(distances)
            self.cost_history.append(hinge_loss)

            for idx, x_i in enumerate(X):
                condition = y[idx] * (np.dot(x_i, self.w) + self.b) >= 1
                if condition:
                    dw = 2 * self.lambda_param * self.w
                    db = 0
                else:
                    dw = 2 * self.lambda_param * self.w - y[idx] * x_i
                    db = -y[idx]

                self.w -= self.lr * dw
                self.b -= self.lr * db

    def get_raw_score(self, X):
        return np.dot(X, self.w) + self.b


class MultiClassSVM:
    def __init__(self, n_classes=3, learning_rate=0.01, lambda_param=0.01, epochs=1000):
        self.n_classes = n_classes
        # Initialize an array of standard Binary SVMs
        self.models = [
            LinearSVM(learning_rate, lambda_param, epochs) for _ in range(n_classes)
        ]

    def fit(self, X, y):
        for i in range(self.n_classes):
            print(f"  -> Training SVM {i+1} (Class {i} vs Rest)...")
            # Convert target: 1 if class matches 'i', -1 otherwise
            y_binary = np.where(y == i, 1, -1)
            self.models[i].fit(X, y_binary)

    def predict(self, X):
        # Gather scores (w*x + b) from all 3 models
        scores = np.zeros((X.shape[0], self.n_classes))
        for i in range(self.n_classes):
```

```python
            scores[:, i] = self.models[i].get_raw_score(X)

        # The predicted class is the one with the highest confidence score
        return np.argmax(scores, axis=1)


# 4. MAIN EXECUTION
def load_all_iris():
    url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
    try:
        df = pd.read_csv(url)
    except Exception as e:
        print(f"Error loading data: {e}")
        return None, None, None
    # Map all 3 classes to 0, 1, 2
    class_names = ["setosa", "versicolor", "virginica"]
    mapper = {name: i for i, name in enumerate(class_names)}
    df["species"] = df["species"].map(mapper)
    # 2 Features for Visualization
    X = df[["petal_length", "petal_width"]].values
    y = df["species"].values
    return X, y, class_names


def main():
    print("=" * 50)
    print(" MULTI-CLASS SVM (One-vs-Rest) CLASSIFICATION")
    print("=" * 50)
    # 1. Load Data
    X, y, class_names = load_all_iris()
    if X is None:
        return
    # 2. Split and Standardize
    X_train, X_test, y_train, y_test = train_test_split_custom(X, y, test_size=0.2)
    X_train_std, mean_val, std_val = standardize(X_train)
    X_test_std, _, _ = standardize(X_test, mean_val, std_val)
    # 3. Train Multi-Class SVM
    print("\nTraining Multi-Class SVM (One-vs-Rest strategy)...")
    ovr_svm = MultiClassSVM(
        n_classes=3, learning_rate=0.01, lambda_param=0.01, epochs=1500
    )
    ovr_svm.fit(X_train_std, y_train)
    # 4. Predict & Evaluate
    y_pred = ovr_svm.predict(X_test_std)
    acc, prec, rec, f1, cm = evaluate_metrics_multiclass(y_test, y_pred)

    print("\n" + "=" * 40)
    print("EVALUATION REPORT (3-Class)")
    print("=" * 40)
    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision (Macro): {prec:.4f}")
    print(f"Recall (Macro):    {rec:.4f}")
    print(f"F1 Score (Macro):  {f1:.4f}")
    print("\nConfusion Matrix:")
    print(cm)
    # 5. Visualizations. Plot all 3 Cost Histories (One for each SVM trained)
    plot_multiclass_cost_history(ovr_svm.models, class_names)

    plot_confusion_matrix(
        cm, class_names=class_names, title="Multi-Class SVM Confusion Matrix"
    )

    plot_multiclass_decision_boundary(
        X_test_std,
        y_test,
        ovr_svm,
        class_names=class_names,
        title="Multi-Class SVM Decision Regions",
    )


if __name__ == "__main__":
    main()
```

3 Results and Discussion

The Multi-Class SVM was trained using $K = 3$ binary classifiers with a learning rate of 0.01, a lambda parameter of 0.01, over 1500 epochs.

3.1 Quantitative Results

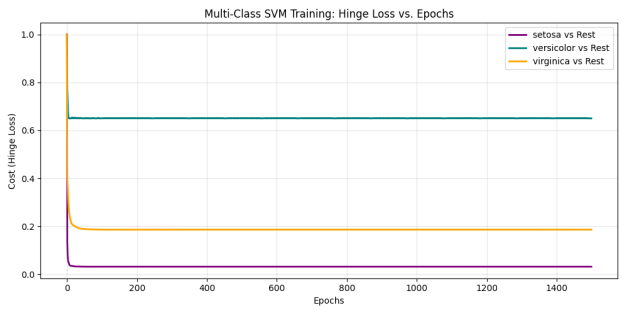
Based on the test split, the model achieved a high degree of accuracy. The evaluation report derived from the confusion matrix is as follows:

```
==================================================
```

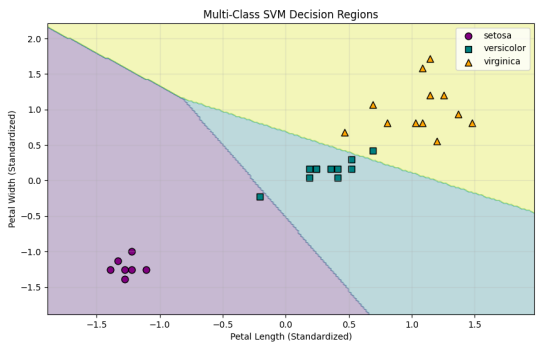
```
 MULTI-CLASS SVM (One-vs-Rest) CLASSIFICATION
==================================================
Loading Iris Dataset...
Training Multi-Class SVM (One-vs-Rest strategy)...
  -> Training SVM 1 (Class 0 vs Rest)...
  -> Training SVM 2 (Class 1 vs Rest)...
  -> Training SVM 3 (Class 2 vs Rest)...

========================================
EVALUATION REPORT (3-Class)
========================================
Accuracy:  0.9333
Precision (Macro): 0.9327
Recall (Macro):    0.9393
F1 Score (Macro):  0.9350

Confusion Matrix:
[[ 7  0  0]
 [ 1  9  1]
 [ 0  0 12]]
```

The confusion matrix (Figure 2c) confirms that *Setosa* and *Virginica* were easily separated from the rest, while *Versicolor* experienced minor misclassifications (one sample confused for Setosa, and one for Virginica). This aligns with the geometric reality of the dataset, where Versicolor sits in the middle of the other two classes.

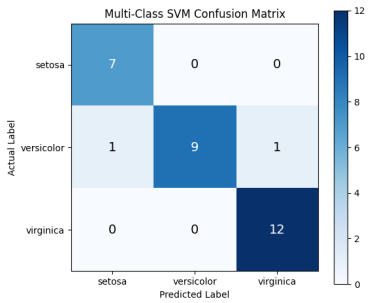
3.2 Visual Analysis



(a) Hinge Loss vs. Epochs



(b) Decision Regions (OvR Boundary)



(c) Multi-Class Confusion Matrix

Figure 2: Multi-Class SVM Performance Overview

- **Cost History (Figure 2a):** The cost function demonstrates distinct convergence behavior for the three models. The *Setosa vs Rest* (purple) and *Virginica vs Rest* (orange) models converge to a lower hinge loss compared to *Versicolor vs Rest* (teal). This mathematically proves that separating the middle cluster from the extremes is harder and yields a higher residual cost.
- **Decision Boundary (Figure 2b):** The OvR strategy successfully creates a non-linear global boundary out of three linear boundaries. The sharp angles where the colored regions meet are the exact points where the maximum confidence score $w^T x + b$ switches from one binary classifier to another.

4 Conclusion

This experiment successfully demonstrated the implementation of a linear Support Vector Machine from first principles. By leveraging the One-vs-Rest (OvR) strategy, the strict binary limitation of the SVM algorithm was overcome, allowing it to accurately classify the 3-class Iris dataset with 93.33% accuracy. The results confirm that minimizing the regularized Hinge Loss function via Stochastic Gradient Descent provides robust, maximum-margin decision boundaries.

References

[1] C. M. Bishop, *Pattern recognition and machine learning.* springer, 2006.

[2] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.